

Spreadsheet Formula Compiler

1. Project Overview

This project is a spreadsheet formula compiler that tokenizes, parses, and analyzes spreadsheet formulas (e.g., =A1 + B2, =SUM(A1:A5)). It transforms raw formula text into an Abstract Syntax Tree (AST) that captures the structure and relationships between operators, operands, and function calls. The compiler implements a complete pipeline from lexical analysis through syntax parsing, with comprehensive error detection at each stage to catch malformed formulas before execution.

2. System Architecture

The compiler follows a classic three-stage pipeline architecture. Each stage is independent and passes structured data to the next: the Lexer converts characters into tokens, the Parser builds an AST from tokens, and the AST is traversed to produce output.

```
Formula String → [Lexer] → Token Stream → [Parser] → AST → Output
                                     ↓
                                     Print / JSON
```

3. Component Design

Scanner / Lexer (Lexer.java + Token.java)

The Lexer performs character-by-character analysis to recognize tokens: numbers, cell references (A1, B5), ranges (A1:A5), operators (+ - * / > < >= <= != ==), and function names (SUM, MAX, MIN, IF). It uses two static Maps to map two-character and single-character operators to their token types, reducing code duplication. It outputs a list of Token objects, each containing type, lexeme, line, and column information for accurate error reporting.

Parser (Parser.java)

The Parser implements recursive descent with operator precedence via method hierarchy: `comparison()` → `additive()` → `multiplicative()` → `primary()`. Each level handles its operators and recursively calls the next lower level, enforcing precedence without precedence climbing. A generic `binaryOp()` helper method eliminates code duplication across the three precedence levels. The Parser also validates semantic constraints such as range ordering (start must come before end).

AST (ASTNode.java)

The AST uses an abstract base class with six concrete node types: `NumberNode`, `CellNode`, `RangeNode`, `BinaryNode`, `FuncNode`, and `FormulaNode` (root). Each node implements `print(int indent)` for tree display and `toJson(int indent)` for JSON serialization. All nodes share the `indent()` helper from the base class to avoid duplication. The hierarchy directly reflects the kinds of values that can appear in formulas.

Main / Driver (Main.java)

Main orchestrates the full pipeline: it reads input (hardcoded test cases or from a file via --file), invokes the Lexer and Parser in sequence, and formats output as both an indented AST tree and JSON. A `hasErrors()` helper method checks and reports errors from both stages with clear diagnostic labels.

4. Grammar Design

The grammar uses precedence levels to enforce correct evaluation order. It is well-suited for recursive descent because each level is a separate method calling the next-lower level. Left-to-right associativity emerges naturally from the while-loop structure in each method.

```
expression    ::= comparison
comparison    ::= additive {(">"|"<"|>="|<="|=="|!=") additive}
additive      ::= multiplicative {("+ "|" -") multiplicative}
multiplicative ::= primary {("*"|" /") primary}
primary       ::= (NUMBER | CELL_REF [":" CELL_REF] | FUNCTION (expression {,
expression}))
```

5. Error Handling Strategy

Lexical errors occur when the Lexer encounters invalid characters (e.g., @, #). These are collected in an errors list with position info: "Invalid char: @ at 1:5". Parsing continues to detect all errors in one pass rather than stopping at the first.

Syntax errors occur when the Parser encounters unexpected token sequences. These are also collected with context, for example: "Expected number, cell reference, function, or '(' when EOF is reached after a dangling operator like =A1+.

6. Design Decisions

- **Handwritten Lexer instead of a generator:** The token set is small and simple. Hand-coding provides clarity, avoids external dependencies, and makes custom error tracking straightforward. The Maps-based approach is more maintainable than generated code for small grammars.
- **Recursive descent instead of table-driven (LR) parsing:** The grammar is left-recursive-free and has low complexity, making recursive descent natural and readable. Each precedence level maps directly to a method, making the code self-documenting and easy to extend without parser generation tools.
- **AST node hierarchy with abstract base class:** Using polymorphism allows uniform traversal and output generation. Each node encapsulates its specific data while implementing a common interface (`print()`, `toJson()`). This simplifies the Main driver and makes adding new output formats straightforward.
- **Range validation at parse time:** The parser checks that range start comes before range end (e.g., rejects A5:A1). This catches a semantic error early before AST traversal, ensuring the AST always represents valid ranges.

7. Known Limitations

- No support for negative literals: Expressions like `=-5` are not recognized. The lexer treats `-` as an operator, not a unary prefix, so negative numbers must be expressed via subtraction (e.g., `=0-5`).
- Limited function set: Only four functions are recognized (SUM, MAX, MIN, IF). Adding new functions requires modifying the FUNCTIONS set in Lexer.java. There is no mechanism for dynamic function registration.
- No named ranges or variables: Formulas can only reference individual cells (A1) or ranges (A1:A5), not named ranges like "SalesTotal" or user-defined variables.
- No evaluation stage: The compiler is a front-end only. It produces an AST but does not evaluate formulas or retrieve actual cell values. A back-end evaluator would need to be added separately.